

Zero-Knowledge Proofs of Possession for n -ary Operations,

and a Self-Hosting Proof Tower

A machine-checked construction in Lean 4

Abstract

This note documents a formalisation, checked by Lean 4 with Mathlib and free of `sorry`, of a zero-knowledge proof-of-possession (ZKPoP) layer built directly on the mathematics developed elsewhere in this project: n -ary addition tables, the integer compositions that populate their fibres, M -ary splitting trees, the orders of the sporadic groups and the divisor levels of those orders, and the self-hosting network model. Four things are proved. First, the classical three-move Σ -protocol over a base of arbitrary size N is complete, specially sound with an explicit extractor, and honest-verifier zero knowledge, and the size of the base determines both the usable challenge space ($N.\text{minFac}$ challenges) and the number of parallel repetitions needed for a given security level. Second, the protocol is lifted from a scalar secret to an n -ary secret, so that a whole cell $(x_1, \dots, x_n)$ of an n -dimensional addition table — equivalently, a weak composition of the cell's value — becomes the witness of a zero-knowledge proof of knowledge; the fibre has $\binom{k+M-1}{M-1}$ witnesses and the transcript hides which one the prover holds. Third, a *claim* is a decidable checker, a Lean proof of the claim's provability is exactly a certificate accepted by that checker, and the protocol turns such a certificate into an argument of knowledge of the proof: the extractor outputs the certificate itself, while the verifier's view is reproducible by a simulator holding nothing. Fourth, the construction is a fixed point — knowledge of a proof of knowledge is again knowledge of the secret — which yields a tower, provable at every level, each level certifying the level below; and the whole apparatus is priced inside the project's self-hosting economy, where the catalogue's verification bill of 1170 rounds per epoch is paid and the network still reaches its self-hosting stage.

Contents

1	Introduction	2
1.1	What this document is	2
1.2	What is being proved in zero knowledge	3
2	The protocol over a base of size N	3
2.1	Definitions	3
2.2	The three properties	3
2.3	How the size of a base fixes its security	4
3	Bases taken from the project's mathematics	5

4	<i>n</i>-ary operations witnessed in zero knowledge	5
4.1	From the addition table to the public value	5
4.2	The <i>n</i> -ary Σ -protocol	6
4.3	Splittings are witnesses	6
4.4	Refinement raises the arity of a witness	7
4.5	Splitting trees	7
5	An argument of knowledge of a Lean proof	8
5.1	Claims and certificates	8
5.2	Committing, extracting, hiding	8
5.3	Packing an <i>n</i> -ary witness into a single certificate	9
5.4	Two worked claims	10
6	The fixed point and the self-hosting tower	10
6.1	Knowledge of a proof of knowledge	10
6.2	The tower	11
7	The zero-knowledge layer inside the self-hosting economy	11
7.1	The cost of a verification	11
7.2	Why proving beats enumerating	11
7.3	The catalogue’s bill, and self-hosting	12
7.4	Paying for the tower	12
8	Scope, and what is <i>not</i> claimed	13
9	Verification	13

1 Introduction

1.1 What this document is

Everything stated below is a theorem of the Lean development in this repository. Displayed mathematics is the informal reading; the accompanying listing is the formal statement, verbatim from the source. The modules are

file	content
<code>RequestProject/ZkPoP.lean</code>	the Σ -protocol over an arbitrary base
<code>RequestProject/ZkPoPBases.lean</code>	twelve bases taken from the project’s own objects
<code>RequestProject/ZkPoPNary.lean</code>	<i>n</i> -ary secrets: table cells and splitting trees as witnesses
<code>RequestProject/ZkPoPProofs.lean</code>	claims, certificates, arguments of knowledge of a proof
<code>RequestProject/ZkPoPSelfHosting.lean</code>	the cost of verifying, inside the self-hosting economy

and the headline results of each are re-checked in `RequestProject/AxiomAudit.lean` (Threads `Q` and `R`), where they are shown to depend only on the three standard axioms `propext`, `Classical.choice`, `Quot.sound`.

1.2 What is being proved in zero knowledge

The object of a proof of possession is a public value y and the assertion “I know a secret behind y ”. In this development the secret ranges over exactly the mathematics the project already contains:

- a residue modulo the order of a sporadic simple group, or modulo the number of cells of an n -ary addition table, of splittings of an integer, of splitting-tree shapes, of divisors at a given level (§3);
- a *tuple*: a cell $(x_1, \dots, x_n)$ of the n -dimensional addition table with a prescribed value, i.e. a weak composition, i.e. a way of splitting a table entry into n parts (§4);
- a *certificate*: a number accepted by a decidable checker — which is to say, a Lean proof of an existential statement, argued in zero knowledge (§5);
- a *proof of knowledge itself*: an accepting transcript pair, giving the fixed point and the self-hosting tower (§6).

2 The protocol over a base of size N

2.1 Definitions

Fix a modulus N , the *base size*, and a public element $g \in \mathbb{Z}/N\mathbb{Z}$.

Definition 2.1. The public value of the secret x is $\text{key}(g, x) = x \cdot g$. A transcript is a triple (a, c, s) and is *accepted* for (g, y) when

$$s \cdot g = a + c \cdot y.$$

The honest prover with secret x and randomness k answers the challenge c with $\text{prove}(g, x, k, c) = (k \cdot g, c, k + c \cdot x)$.

```
structure Transcript (N : ℕ) where
  commit : ZMod N
  challenge : ZMod N
  response : ZMod N

def key (g x : ZMod N) : ZMod N := x * g

def Verify (g y : ZMod N) (t : Transcript N) : Prop :=
  t.response * g = t.commit + t.challenge * y

def prove (g x k c : ZMod N) : Transcript N := {k * g, c, k + c * x}
```

2.2 The three properties

Theorem 2.2 (Completeness). *For all g, x, k, c the honest transcript is accepted for the public value $\text{key}(g, x)$.*

```
theorem completeness (g x k c : ZMod N) : Verify g (key g x) (prove g x k c)
```

Theorem 2.3 (Special soundness, with an explicit extractor). *Let (a, c_1, s_1) and (a, c_2, s_2) both be accepted for (g, y) and suppose $c_1 - c_2$ is a unit of $\mathbb{Z}/N\mathbb{Z}$. Then*

$$x = (c_1 - c_2)^{-1}(s_1 - s_2)$$

satisfies $\text{key}(g, x) = y$. Answering two such challenges from one commitment therefore requires possession of a secret.

```
noncomputable def extract (c1 c2 s1 s2 : ZMod N) (hu : IsUnit (c1 - c2)) : ZMod N :=
  ((hu.unit⁻¹ : (ZMod N)ˣ) : ZMod N) * (s1 - s2)

theorem extract_key (g y a c1 c2 s1 s2 : ZMod N) (hu : IsUnit (c1 - c2))
  (h1 : Verify g y (a, c1, s1)) (h2 : Verify g y (a, c2, s2)) :
  key g (extract c1 c2 s1 s2 hu) = y
```

Theorem 2.4 (Honest-verifier zero knowledge). *The simulator $\text{simulate}(g, y, c, s) = (s \cdot g - c y, c, s)$, which knows no secret, produces only accepted transcripts, and the substitution $s = k + c x$ — a bijection of the randomness — turns it into the honest prover. Over a finite base the two therefore range over the same transcripts with the same multiplicities.*

```
theorem simulate_eq_prove (g x k c : ZMod N) :
  simulate g (key g x) c (k + c * x) = prove g x k c

theorem zeroKnowledge_image (g x c : ZMod N) :
  (Finset.univ.image fun k : ZMod N => prove g x k c)
  = Finset.univ.image fun s : ZMod N => simulate g (key g x) c s

theorem zeroKnowledge_card (g x c : ZMod N) (t : Transcript N) :
  (Finset.univ.filter fun k : ZMod N => prove g x k c = t).card
  = (Finset.univ.filter fun s : ZMod N => simulate g (key g x) c s = t).card
```

2.3 How the size of a base fixes its security

Special soundness needs $c_1 - c_2$ invertible. Distinct residues below the least prime factor of N always have that property, so a base of size N supports a challenge space of size $N.\text{minFac}$ and a one-shot soundness error of $1/N.\text{minFac}$.

```
theorem sub_isUnit_of_lt_minFac {N i j : ℕ} (hi : i < N.minFac) (hj : j < N.minFac)
  (hne : i ≠ j) : IsUnit ((i : ZMod N) - (j : ZMod N))

def challenges (N : ℕ) : Finset ℕ := Finset.range N.minFac
```

The number of parallel repetitions needed to bring the error below $2^{-\text{bits}}$ is

$$\text{rounds}(N, \text{bits}) = \lfloor \log_{N.\text{minFac}} 2^{\text{bits}} \rfloor + 1,$$

and this is proved to be the least such number:

```
def rounds (N bits : ℕ) : ℕ := Nat.log N.minFac (2 ^ bits) + 1

theorem pow_minFac_rounds {N : ℕ} (hN : 1 < N.minFac) (bits : ℕ) :
  2 ^ bits < N.minFac ^ rounds N bits

theorem rounds_le {N : ℕ} (bits t : ℕ) (ht : 2 ^ bits < N.minFac ^ t) : rounds N bits ≤ t
```

3 Bases taken from the project's mathematics

Every number the project produces can serve as a base. Twelve of them are instantiated, each with a proof of provenance (that the size really is the quantity claimed), of its challenge count, and of the rounds it needs at the 128-bit level.

object	base size N	challenges $N.\text{minFac}$	rounds for 128 bits
order of M_{11}	7920	2	129
order of M_{12}	95040	2	129
order of J_1	175560	2	129
order of J_2	604800	2	129
cells of the ternary addition table up to 9	1000	2	129
level 4 of the divisor table of $ M_{11} $	12	2	129
binary split-tree shapes with 5 nodes	42	2	129
splittings of 5 into 3 parts	21	3	81
splittings of 6 into 2 parts	7	7	46
bounded splittings of 3 into 3 parts of size ≤ 2	7	7	46
binary split-tree shapes with 3 nodes	5	5	56
level 3 of the divisor table of $ M_{11} $	11	11	38

The table is itself a theorem: `ZkPoPBases.report_correct` states that the list of rows above is the list of measurements of the twelve bases, and `ZkPoPBases.all_bases_protocol` states that each of them supports the protocol. The prime bases use the whole residue ring as challenge space; the composite ones — the group orders in particular, all even — admit only a binary challenge, which is exactly why they need 129 rounds where a base of size 11 needs 38.

4 n -ary operations witnessed in zero knowledge

4.1 From the addition table to the public value

The project's central object is the n -ary addition operation and its table,

$$\text{naryAdd}(x_1, \dots, x_n) = \sum_{i=1}^n x_i,$$

whose graph is an n -dimensional hypercube of cells, and whose fibre over a value k is the set of *weak compositions* (splittings)

$$\text{splits}(M, k) = \left\{ (y_1, \dots, y_M) \in \mathbb{N}^M : \sum_i y_i = k \right\}, \quad \#\text{splits}(M, k) = \binom{k+M-1}{M-1}.$$

To witness such a cell we replace the scalar public value $x \cdot g$ by the n -ary linear form

$$\text{linComb}(g, x) = \sum_{i=1}^n x_i g_i,$$

which for the all-ones coefficient vector is precisely the reduction of `naryAdd` modulo the base.

```

def linComb (g x : Fin n → ZMod N) : ZMod N := ∑ i, x i * g i

def ones (N n : ℕ) : Fin n → ZMod N := fun _ => 1

theorem linComb_ones_cast (w : Fin n → ℕ) :
  linComb (ones N n) (fun i => ((w i : ℕ) : ZMod N)) = ((NaryTable.naryAdd w : ℕ) : ZMod N)

```

4.2 The n -ary Σ -protocol

The prover commits to a random *tuple* k , answers a challenge c componentwise by $s_i = k_i + c x_i$, and the verifier still checks a single equation:

$$\sum_i s_i g_i = a + c y.$$

```

structure NTranscript (N n : ℕ) where
  commit : ZMod N
  challenge : ZMod N
  response : Fin n → ZMod N

def VerifyN (g : Fin n → ZMod N) (y : ZMod N) (t : NTranscript N n) : Prop :=
  linComb g t.response = t.commit + t.challenge * y

def proveN (g x k : Fin n → ZMod N) (c : ZMod N) : NTranscript N n :=
  {linComb g k, c, fun i => k i + c * x i}

```

Theorem 4.1 (The n -ary protocol). *The n -ary protocol is complete; two accepting transcripts sharing a commitment, with challenges differing by a unit, extract the whole tuple $x_i = (c_1 - c_2)^{-1}(s_{1,i} - s_{2,i})$ with $\text{linComb}(g, x) = y$; and it is honest-verifier zero knowledge, the simulator's transcripts coinciding with the prover's.*

```

theorem completeness_nary (g x k : Fin n → ZMod N) (c : ZMod N) :
  VerifyN g (keyN g x) (proveN g x k c)

theorem extract_nary (g : Fin n → ZMod N) (y a 1c 2c : ZMod N) (1s 2s : Fin n → ZMod N)
  (hu : IsUnit (1c - 2c)) (1h : VerifyN g y (a, 1c, 1s)) (2h : VerifyN g y (a, 2c, 2s)) :
  keyN g (extractVec 1c 2c 1s 2s hu) = y

theorem zeroKnowledge_nary_image (g x : Fin n → ZMod N) (c : ZMod N) :
  (Finset.univ.image fun k : Fin n → ZMod N => proveN g x k c)
  = Finset.univ.image fun s : Fin n → ZMod N => simulateN g (keyN g x) c s

```

4.3 Splittings are witnesses

Theorem 4.2 (A cell of the table is a witness). *If $w \in \text{splits}(M, k)$ then $\text{keyN}(\mathbf{1}, w) = k$ in the base, so the holder of any splitting of k into M parts convinces the verifier that the M -ary table takes the value k ; conversely two accepting transcripts yield a tuple over the base summing to k .*

```

theorem splits_witness {M k : ℕ} {w : Fin M → ℕ} (hw : w ∈ NaryTable.splits M k) :
  keyN (ones N M) (ofSplit N w) = ((k : ℕ) : ZMod N)

```

```

theorem splits_completeness {M k : ℕ} {w : Fin M → ℕ} (hw : w ∈ NaryTable.splits M k)
  (r : Fin M → ZMod N) (c : ZMod N) :
  VerifyN (ones N M) ((k : ℕ) : ZMod N) (proveN (ones N M) (ofSplit N w) r c)

theorem splits_soundness {M k : ℕ} (a₁ c₂ c : ZMod N) (is₂s : Fin M → ZMod N)
  (hu : IsUnit (₁c - ₂c))
  (ih : VerifyN (ones N M) ((k : ℕ) : ZMod N) {a, ₁c, ₁}s)
  (zh : VerifyN (ones N M) ((k : ℕ) : ZMod N) {a, ₂c, ₂}s) :
  ∃ x : Fin M → ZMod N, ∑ i, x i = ((k : ℕ) : ZMod N)

```

Theorem 4.3 (How much is hidden). *The statement “the M -ary table takes the value k ” has exactly $\text{multichoose}(M, k) = \binom{k+M-1}{M-1}$ witnesses, and any two of them produce exactly the same set of transcripts: the proof hides which splitting the prover holds.*

```

theorem card_witnesses (M k : ℕ) : (NaryTable.splits M k).card = Nat.multichoose M k

theorem witness_indistinguishable [NeZero N] {M k : ℕ} {w w' : Fin M → ℕ}
  (hw : w ∈ NaryTable.splits M k) (hw' : w' ∈ NaryTable.splits M k) (c : ZMod N) :
  (Finset.univ.image fun r : Fin M → ZMod N => proveN (ones N M) (ofSplit N w) r c)
  = Finset.univ.image fun r : Fin M → ZMod N => proveN (ones N M) (ofSplit N w') r c

```

For example, at $M = 2$, $k = 3$ the fibre is $\{(0, 3), (1, 2), (2, 1), (3, 0)\}$: four witnesses, one public statement, one transcript distribution.

4.4 Refinement raises the arity of a witness

Replacing each input of an n -ary sum by a splitting of it into M parts is the project’s “refinement” move; it raises the arity to $n \cdot M$ without changing the value. In the protocol this says that the *same* public statement can be proved at any arity: a witness at arity n refines to a witness at arity nM .

```

theorem refine_witness {n M : ℕ} (x : Fin n → ℕ) (y : Fin n → Fin M → ℕ)
  (hy : ∀ i, y i ∈ NaryTable.splits M (x i)) :
  keyN (ones N (n * M)) (ofSplit N (fun p : Fin (n * M) =>
    y (finProdFinEquiv.symm p).1 (finProdFinEquiv.symm p).2))
  = keyN (ones N n) (ofSplit N x)

```

4.5 Splitting trees

Iterating the splitting produces M -ary trees whose value is the sum of their leaves. Each internal node of such a tree is itself an M -ary witness — the children’s values form a splitting of the node’s value — so a proof of knowledge of a whole tree is assembled node by node, one Σ -protocol instance per node.

```

theorem tree_leaves_witness {M : ℕ} (t : SplitTree M) :
  ((t.value : ℕ) : ZMod N) = (t.leaves.map (fun k : ℕ => ((k : ℕ) : ZMod N))).sum

theorem tree_node_witness {M : ℕ} (f : Fin M → SplitTree M) :
  (fun i => (f i).value) ∈ NaryTable.splits M (SplitTree.node f).value

theorem tree_node_completeness {M : ℕ} (f : Fin M → SplitTree M) (r : Fin M → ZMod N)
  (c : ZMod N) :

```

```
VerifyN (ones N M) (((SplitTree.node f).value : ℕ) : ZMod N)
  (proveN (ones N M) (ofSplit N (fun i => (f i).value)) r c)
```

5 An argument of knowledge of a Lean proof

5.1 Claims and certificates

Definition 5.1. A *claim* is a name together with a decidable checker $\text{code} : \mathbb{N} \rightarrow \text{Bool}$. The claim is *provable* when $\exists w, \text{code}(w) = \text{true}$.

```
structure Claim where
  name : String
  code : ℕ → Bool

def Claim.Provable (C : Claim) : Prop := ∃ w, C.code w = true
```

This is the pivot of the whole construction. A Lean proof of C.Provable is a certificate w with $\text{code}(w) = \text{true}$, and conversely: the two are the same data, related by $\text{Claim.provable_of_code}$ in one direction and by $\text{Claim.someWitness_spec}$ in the other. Consequently, to prove knowledge of w in zero knowledge is to prove knowledge of a Lean proof, without publishing the proof.

```
theorem Claim.provable_of_code {C : Claim} {w : ℕ} (h : C.code w = true) : C.Provable

noncomputable def Claim.someWitness (C : Claim) : ℕ := if h : C.Provable then h.choose else 0

theorem Claim.someWitness_spec {C : Claim} (h : C.Provable) : C.code C.someWitness = true
```

5.2 Committing, extracting, hiding

The commitment of a certificate is its residue $\text{certKey}(N, w) = w \bmod N$; the protocol is run at $g = 1$.

Theorem 5.2 (Completeness, extraction, zero knowledge for proofs). *An honest holder of a certificate always convinces the verifier. Two accepting transcripts with challenges differing by a unit make the extractor output the certificate itself — so the extracted value is again accepted by the checker, i.e. is a Lean proof of the claim. And the transcripts are exactly those of a simulator holding no certificate at all, so the verifier learns nothing beyond the commitment.*

```
def certKey (N w : ℕ) : ZMod N := ((w : ℕ) : ZMod N)

theorem proof_completeness (C : Claim) {w : ℕ} (_hw : C.code w = true) (r c : ZMod N) :
  Verify (1 : ZMod N) (certKey N w) (prove (1 : ZMod N) ((w : ℕ) : ZMod N) r c)

theorem extract_proof [NeZero N] {w : ℕ} (hwN : w < N) (a₁ c₂ c₁ s₂ s : ZMod N)
  (hu : IsUnit (c₁ - c₂))
  (₁h : Verify (1 : ZMod N) (certKey N w) (a₁, c₁, ₁s))
  (₂h : Verify (1 : ZMod N) (certKey N w) (a₁, c₂, ₂s)) :
  (extract c₁ c₂ c₁ s₂ s hu).val = w

theorem extract_certifies [NeZero N] (C : Claim) {w : ℕ} (hw : C.code w = true) (hwN : w < N)
  (a₁ c₂ c₁ s₂ s : ZMod N) (hu : IsUnit (c₁ - c₂))
```



```

(1h : Verify (1 : ZMod N) (certKey N w) (a, 1c, 1)s)
(2h : Verify (1 : ZMod N) (certKey N w) (a, 2c, 2)s) :
C.code (extract 1c 2c 1s 2s hu).val = true ∧ C.Provable

theorem proof_zero_knowledge [NeZero N] (w : N) (c : ZMod N) :
(Finset.univ.image fun r : ZMod N => prove (1 : ZMod N) (certKey N w) r c)
= Finset.univ.image fun s : ZMod N => simulate (1 : ZMod N) (certKey N w) c s

```

Remark 5.3. The hypothesis $w < N$ in `extract_proof` is the statement that the base is large enough to hold the certificate; it is what makes the commitment injective on certificates, and it is the only place where the size of the base enters the extraction argument.

5.3 Packing an n -ary witness into a single certificate

The claims above take a single number as certificate, while the n -ary statements of §4 have tuples as witnesses. The two are reconciled by base- $(B+1)$ digits: a tuple of M parts, each at most B , is packed into one number below $(B+1)^M$, and unpacking recovers it.

$$\text{encodeVec}(x) = \sum_{i=0}^{M-1} x_i (B+1)^i, \quad \text{digitVec}(w)_i = \left\lfloor \frac{w}{(B+1)^i} \right\rfloor \bmod (B+1).$$

```

def digitVec (B M w : N) : Fin M → N := fun i => (w / (B + 1) ^ (i : N)) % (B + 1)

def encodeVec (B : N) : ∀ {M : N}, (Fin M → N) → N
| 0, _ => 0
| (_ + 1), x => x 0 + (B + 1) * encodeVec B (fun i => x i.succ)

theorem digitVec_encodeVec (B : N) : ∀ {M : N} (x : Fin M → N), ∀( i, x i ≤ B) →
digitVec B M (encodeVec B x) = x

theorem encodeVec_lt (B : N) : ∀ {M : N} (x : Fin M → N), ∀( i, x i ≤ B) →
encodeVec B x < (B + 1) ^ M

```

This makes the n -ary statement a claim in the above sense: “I know a splitting of k into M parts, each at most B ”, with a single-number certificate.

```

def narySplitClaim (B M k : N) : Claim where
  name := "a splitting of k into M parts of size at most B"
  code := fun w => decide ∑( i, digitVec B M w i = k)

theorem narySplitClaim_provable {B M k : N} {x : Fin M → N} (hx : ∀ i, x i ≤ B)
(hsum : x ∈ NaryTable.splits M k) : (narySplitClaim B M k).Provable

theorem narySplitClaim_sound {B M k w : N} (hw : (narySplitClaim B M k).code w = true) :
digitVec B M w ∈ NaryTable.splits M k ∧ ∀ i, digitVec B M w i ≤ B

theorem narySplit_extract [NeZero N] {B M k w : N}
(hw : (narySplitClaim B M k).code w = true) (hwN : w < N)
(a 1c 2c 1s 2s : ZMod N) (hu : IsUnit (1c - 2c))
(1h : Verify (1 : ZMod N) (certKey N w) (a, 1c, 1)s)
(2h : Verify (1 : ZMod N) (certKey N w) (a, 2c, 2)s) :
digitVec B M (extract 1c 2c 1s 2s hu).val ∈ NaryTable.splits M k

```

The last statement is the n -ary argument of knowledge in its final form: whoever answers two challenges hands the extractor a genuine cell of the M -ary addition table of value k — a cell the verifier never sees.

5.4 Two worked claims

- $(1, 2, 0)$ splits 3 into three parts of size at most 2, so the ternary claim of value 3 has a proof: `narySplitClaim_three`.
- $|M_{11}| = 7920$ is composite, so the factoring claim about the order of the Mathieu group M_{11} — a number the project derives elsewhere — has a proof, conveyed by the protocol without revealing the factor: `factorClaim_M11`.

```
theorem narySplitClaim_three : (narySplitClaim 2 3 3).Provable

def factorClaim (n : ℕ) : Claim where
  name := "a nontrivial factor of n"
  code := fun d => decide (1 < d ∧ d < n ∧ d | n)

theorem factorClaim_M11 : (factorClaim 7920).Provable
theorem factorClaim_M11_provenance : ("M11", 7920) ∈ SporadicOrders.sporadic
```

6 The fixed point and the self-hosting tower

6.1 Knowledge of a proof of knowledge

Consider the claim whose certificates are *accepting transcript pairs* for a published statement (g, y) , with the two challenges 0 and 1. In every base $0 - 1 = -1$ is a unit, so a certificate for this claim is precisely what the extractor of §2 consumes.

```
def encode3 (a es is : ZMod N) : ℕ := Nat.pair a.val (Nat.pair es.val is.val)

def transcriptPairClaim (N : ℕ) (g y : ZMod N) : Claim where
  name := "an accepting transcript pair for (g, y)"
  code := fun w =>
    decide (Verify g y ((decode3 N w).1, 0, (decode3 N w)).2.1 ∧
      Verify g y ((decode3 N w).1, 1, (decode3 N w)).2.2)
```

Theorem 6.1 (Fixed point). *For every base and every published statement,*

$$“I \text{ know a proof of knowledge for } (g, y)” \iff \exists x, \text{key}(g, x) = y,$$

so the claim of knowledge of a proof is equi-provable with the original claim of knowledge.

```
theorem transcriptPair_iff [NeZero N] (g y : ZMod N) :
  (transcriptPairClaim N g y).Provable ↔ ∃ x, key g x = y

theorem proof_of_proof_idempotent [NeZero N] (g y : ZMod N) :
  (transcriptPairClaim N g y).Provable ↔ (secretClaim N g y).Provable
```

The two directions are the two halves of the protocol. Left to right is special soundness: a transcript pair extracts a secret. Right to left is completeness: from a secret one exhibits the pair $(0, 0, 0)$ and $(0, 1, x)$, both accepted, sharing the commitment 0.

6.2 The tower

Iterate: level $j+1$ publishes the commitment of the certificate used at level j , and claims knowledge of a proof for that. Formally,

$$y_0 = y, \quad y_{j+1} = \text{key}(g, w_j), \quad w_j = \text{a certificate of the level-}j \text{ claim,}$$

and every level is provable as soon as the ground statement has a secret.

```

noncomputable def towerPublic (N : ℕ) (g y : ZMod N) : ℕ → ZMod N
| 0 => y
| j + 1 => key g (((transcriptPairClaim N g (towerPublic N g y j)).someWitness : ℕ) : ZMod N)

noncomputable def towerClaim (N : ℕ) (g y : ZMod N) (j : ℕ) : Claim :=
  transcriptPairClaim N g (towerPublic N g y j)

theorem towerClaim_provable [NeZero N] (g y : ZMod N) (h : ∃ x, key g x = y) (j : ℕ) :
  (towerClaim N g y j).Provable

theorem tower_commits_previous_proof [NeZero N] (g y : ZMod N) (h : ∃ x, key g x = y) (j : ℕ) :
  (towerClaim N g y j).code (towerClaim N g y j).someWitness = true ∧
  towerPublic N g y (j + 1)
  = key g (((towerClaim N g y j).someWitness : ℕ) : ZMod N)

```

This is the self-hosting statement at the level of proofs: the layer proves statements about its own proofs, for ever, and each level's published value is exactly the commitment of a certificate the level below accepts. Because of the fixed point, the tower neither strengthens nor weakens what is being asserted — it only re-hosts it.

7 The zero-knowledge layer inside the self-hosting economy

7.1 The cost of a verification

A round of the M -ary protocol puts $M + 2$ base elements on the wire (a commitment, a challenge, M responses). Priced in the network's five resource classes (compute, storage, network, proof, data) under the flat tariff, checking t rounds costs

$$\text{zkCost}(M, t) = t(2M + 5).$$

```

def transcriptSize (M : ℕ) : ℕ := M + 2

def zkUsage (M rounds : ℕ) : Usage where
  compute := rounds * transcriptSize M
  storage := 0
  network := rounds * transcriptSize M
  proof := rounds
  data := 0

theorem zkCost_flat (M rounds : ℕ) : zkCost flatTariff M rounds = rounds * (2 * M + 5)

```

7.2 Why proving beats enumerating

The alternative to proving knowledge of one witness is transporting the witness set. For $M \geq 2$ the fibre of the M -ary table over k has at least $k + 1$ elements, so the enumeration bill is unbounded in k , while the verification bill is a fixed function of the arity and the round count.

```

theorem card_splits_ge_succ {M : ℕ} (hM : 1 ≤ M) (k : ℕ) :
  k + 1 ≤ (NaryTable.splits (M + 1) k).card

theorem enumeration_unbounded {M : ℕ} (hM : 1 ≤ M) (budget : ℕ) :
  ∃ k, budget < enumerationCost flatTariff (M + 1) k

theorem zk_cheaper_than_enumeration {M rounds k : ℕ} (hM : 1 ≤ M)
  (hk : zkCost flatTariff (M + 1) rounds ≤ k) :
  zkCost flatTariff (M + 1) rounds < enumerationCost flatTariff (M + 1) k

```

7.3 The catalogue’s bill, and self-hosting

Re-proving all twelve objects of §3 at 128-bit soundness costs

$$7 \cdot 129 + 81 + 46 + 46 + 56 + 38 = 1170$$

rounds per epoch. In the project’s capacity recursion $c_{t+1} = c_t + \text{yield} \cdot c_t - \text{upkeep}$ (truncated at zero), a network whose yield covers that bill still reaches the self-hosting stage within thirty epochs; one whose yield does not is exhausted after at most c_0 epochs. Both sides are theorems.

```

def zkUpkeep : ℕ := (ZkPoPBases.bases.map fun B => B.rounds 128).sum

theorem zkUpkeep_eq : zkUpkeep = 1170

theorem zk_network_self_hosts :
  stageOf (capUp 1170 2 zkUpkeep 30) = Stage.selfHosting

theorem zk_network_collapses {ec : ℕ} (h : ec < zkUpkeep) :
  capUp ec 1 zkUpkeep ec = 0

```

7.4 Paying for the tower

Each level of the tower of §6 is a scalar proof of knowledge, so it costs $7t$ credits to check under the flat tariff, independently of its depth; the bill for the first $j + 1$ levels is $(j + 1) \cdot 7t$, and a network whose yield covers its upkeep can pay it, because capacity at least doubles every epoch while the bill grows linearly.

```

def levelCost (t : Tariff) (rounds : ℕ) : ℕ := zkCost t 1 rounds
theorem levelCost_flat (rounds : ℕ) : levelCost flatTariff rounds = 7 * rounds

def towerBill (t : Tariff) (rounds j : ℕ) : ℕ := (j + 1) * levelCost t rounds

theorem tower_bill_affordable {ec yield upkeep : ℕ} (eh : 1 ≤ ec)
  (hcov : ec + upkeep ≤ yield * ec) (t : Tariff) (rounds j : ℕ) :
  ∃ epoch, towerBill t rounds j ≤ capUp ec yield upkeep epoch

```

Theorem 7.1 (The loop closes). *For a published statement that really has a secret behind it: every level of the proof tower is provable, its certificate is accepted by the checker of the level below, the level’s published value is the commitment of that certificate, and the network — which is simultaneously paying the catalogue’s 1170-round bill each epoch and is self-hosting by epoch thirty — can afford to verify the whole tower.*

```

theorem selfhosting_tower_affordable {N : ℕ} [NeZero N] (g y : ZMod N) (h : ∃ x, key g x = y)
  (t : Tariff) (rounds j : ℕ) :
  (ZkPoPProofs.towerClaim N g y j).Provable ∧
  (ZkPoPProofs.towerClaim N g y j).code
  (ZkPoPProofs.towerClaim N g y j).someWitness = true ∧
  ZkPoPProofs.towerPublic N g y (j + 1)
  = key g (((ZkPoPProofs.towerClaim N g y j).someWitness : ℕ) : ZMod N) ∧
  ∃ epoch, towerBill t rounds j ≤ capUp 1170 2 zkUpkeep epoch

theorem zk_self_hosting_closure {N : ℕ} [NeZero N] (g y : ZMod N) (h : ∃ x, key g x = y) (j : ℕ) :
  stageOf (capUp 1170 2 zkUpkeep 30) = Stage.selfHosting ∧
  (ZkPoPProofs.towerClaim N g y j).Provable ∧
  ∃ epoch, towerBill flatTariff 129 j ≤ capUp 1170 2 zkUpkeep epoch

```

8 Scope, and what is *not* claimed

The formalisation is of an information-theoretic, algebraic Σ -protocol over $\mathbb{Z}/N\mathbb{Z}$, in the standard idealised form: completeness, special soundness with an explicit extractor, and honest-verifier zero knowledge as an exact equality of transcript sets and multiplicities. In particular:

- No computational hardness assumption is used, and none is proved. Hiding here is exact (the transcript distribution is independent of the witness), while binding is the arithmetic statement that the extractor recovers a preimage; the commitment $x \mapsto x \cdot g$ is not claimed to be one-way.
- The soundness error accounting is the combinatorial one: a base of size N offers $N.\text{minFac}$ challenges whose pairwise differences are units, and $\text{rounds}(N, \text{bits})$ is proved to be the least number of parallel repetitions driving $N.\text{minFac}^{-t}$ below $2^{-\text{bits}}$.
- “Knowledge of a Lean proof” is knowledge of a certificate accepted by a decidable checker. This is faithful for existential claims — where a proof and a witness are the same data — and is the sense in which the tower of §6 is self-hosting. It is not a reflection of Lean’s own proof terms inside Lean.
- The economic statements are theorems about the project’s arithmetic model of capacity, tariffs and upkeep, not empirical claims.

9 Verification

The development is sorry-free. From the repository root,

```

lake build RequestProject.ZkPoP RequestProject.ZkPoPBases RequestProject.ZkPoPNary
lake build RequestProject.ZkPoPProofs RequestProject.ZkPoPSelfHosting
lake build RequestProject.AxiomAudit

```

rebuilds the layer and prints the axiom dependencies of every headline theorem quoted above; each depends only on `propext`, `Classical.choice` and `Quot.sound`.